

BookMyShow Ticketing

Normalized theatre, showtime, seat inventory, timed holds, bookings, payments, and webhook idempotency.

Submission document | MySQL 8.0.16+ | Timestamps stored in UTC

Design boundary: InnoDB is the authority for seat ownership. Redis may accelerate availability hints, but a Redis key alone never confirms a hold or booking.

Entities and Tables

Table	Key attributes	Role
users	user_id PK, email UQ, full_name, created_at	Customer placing a booking.
movies	movie_id PK, title, language_code, duration_minutes, certificate	Movie metadata.
theatres	theatre_id PK, name, address, city, time_zone	Venue and local date context.
screens	screen_id PK, theatre_id FK, name; UQ(theatre_id,name)	Auditorium in a theatre.
seats	seat_id PK, screen_id FK, row_label, seat_number, seat_type; UQ(screen,row,number)	Stable physical seat map.
showtimes	showtime_id PK, screen_id FK, movie_id FK, starts_at_utc, ends_at_utc, status	Movie scheduled in a screen.
show_seats	PK(showtime_id,seat_id), price	Published per-show inventory and stable lock row.
bookings	booking_id PK, showtime_id FK, user_id FK, status, hold_expires_at, total_amount, currency	One customer's attempt/order for a show.
booking_seats	booking_seat_id PK, booking_id FK, seat_id FK, price_at_booking; UQ(booking_id,seat_id)	Historical requested-seat and price snapshot.
seat_allocations	PK(showtime_id,seat_id), booking_seat_id UQ, state, hold_expires_at	Current holds/booked seats only. No row means unallocated.
payments	payment_id PK, booking_id FK, provider IDs, idempotency_key, amount, currency, status	Payment attempts; unique constraints prevent duplicate success.
payment_webhook_events	webhook_event_id PK, provider, provider_event_id UQ, payment_id FK, event type, JSON payload, status	Durable deduplicated webhook inbox.
users	user_id PK, email UQ, full_name, created_at	Customer placing a booking.
movies	movie_id PK, title, language_code, duration_minutes, nullable certificate	Movie metadata.
theatres	theatre_id PK, name, address, city, time_zone	Venue and local date context.
screens	screen_id PK, theatre_id FK, name; UQ(theatre_id,name)	Auditorium in a theatre.
seats	seat_id PK, screen_id FK, row_label, seat_number, seat_type; UQ(screen,row,number)	Stable physical seat map.
showtimes	showtime_id PK, screen_id FK, movie_id FK, starts_at_utc, ends_at_utc, status	Movie scheduled in a screen.
show_seats	showtime_id PK part, seat_id PK part, price	Published per-show inventory and stable lock row.
bookings	booking_id PK, showtime_id FK, user_id FK, status, nullable hold_expires_at, total_amount, currency, created_at, updated_at	One customer's attempt/order for a show.

Table	Key attributes	Role
booking_seats	booking_seat_id PK, booking_id FK, seat_id FK, price_at_booking, created_at; UQ(booking_id,seat_id)	Historical requested-seat and price snapshot.
seat_allocations	showtime_id PK part, seat_id PK part, booking_seat_id UQ/FK, state, nullable hold_expires_at	Current holds/booked seats only. No row means unallocated.
payments	payment_id PK, booking_id FK, provider, nullable provider_payment_id, idempotency_key, amount, currency, status, generated nullable successful_booking_id, created_at, updated_at	Payment attempts; unique constraints prevent duplicate success.
payment_webhook_events	webhook_event_id PK, provider, provider_event_id UQ, nullable payment_id FK, event_type, processing_status, JSON payload, received_at, nullable processed_at	Durable deduplicated webhook inbox.

The executable DDL and fixtures are in `sql/02_create_tables.sql` and `sql/03_sample_data.sql`.

Representative sample rows

Table	Sample record
users	(1, anaya@example.com, Anaya Rao)
movies	(101, The Last Monsoon, en, 142 min, UA)
theatres	(201, Orion Screens, Bengaluru, Asia/Kolkata)
screens / seats	(301, theatre 201, Screen 1) / (401, screen 301, A1, PREMIUM)
showtimes	(1001, screen 301, movie 101, 2026-10-01 13:30-15:52 UTC)
bookings	(5001, show 1001, user 1, CONFIRMED, INR 600.00)
show_seats / booking_seats	(1001, seat 401, INR 300.00) / (5011, booking 5001, seat 401, INR 300.00)
seat_allocations	(show 1001, seat 401, booking-seat 5011, BOOKED)
payments	(6001, booking 5001, DEMO_PAY, SUCCEEDED, INR 600.00)
payment_webhook_events	(7001, DEMO_PAY, demo-event-5001-paid, payment.succeeded, PROCESSED)

Normalization

- **1NF:** Attributes are scalar; seat selections are rows, not lists stored on a booking.
- **2NF:** Descriptive values depend on their full candidate key. Seat and show details are separate from booking-seat records.
- **3NF:** Theatre data lives in theatres; screens reference theatres; showtimes reference screens and movies. No theatre or movie description is copied into a showtime.
- **BCNF:** Entity determinants are candidate keys. Inventory uses (showtime, seat); allocations have candidate keys (showtime, seat) and booking_seat_id. booking_seats does not repeat showtime, which is determined by its booking.

Insert/update triggers verify that inventory seats belong to the scheduled screen, booking-seat snapshots exist in the booking's show, and allocation owners match the show and seat. The offered price and the historical price are different facts: future price changes cannot rewrite an existing booking.

Seat Holds and Concurrency

1. Create a `PENDING` booking with a UTC expiry, typically five minutes ahead.
2. Lock the selected `show_seats` rows using `SELECT ... FOR UPDATE`, in ascending seat order for multi-seat requests.
3. Insert a `booking_seats` price snapshot and then a `seat_allocations` row. Release any expired allocation in the same transaction before reclaiming it.
4. Require one inserted allocation per requested seat. The allocation primary key prevents double ownership. If any seat conflicts, roll back the whole transaction.
5. Store the total from locked inventory prices and commit. Return a successful hold only after commit.

An expiry worker locks the pending booking and its inventory rows, deletes only expired allocation rows still held by that booking, and marks the booking `EXPIRED` atomically. Booking-seat history remains for auditability.

Expiry and payment race

Expiry and payment confirmation lock the booking and seat inventory in the same order. Payment success is accepted only while the booking is pending, before expiry, and every requested seat still has a matching held allocation. The transaction marks the booking confirmed and allocations booked. If expiry wins, a late successful charge is refunded or compensated; it must not reclaim the seat.

Idempotent Payment Webhooks

1. Verify provider signature and timestamp before accepting an event.
2. Insert into `payment_webhook_events`; unique (provider, provider_event_id) makes retries idempotent.
3. In one transaction, lock event/payment/booking, validate payment ID, amount, currency, and booking state, then update payment, booking, allocations, and event status.
4. Dispatch ticket, email, and refund side effects through an outbox or independently idempotent workers.

The payment idempotency key handles retried payment creation. A generated unique successful-booking key permits at most one successful payment per booking while allowing failed attempts.

P2: Shows for Theatre and Date

Convert local midnight and the following local midnight in the theatre's IANA time zone to UTC in the application. Pass those instants as a half-open interval; resolving both boundaries separately handles 23- and 25-hour DST days.

```
SELECT sh.showtime_id, m.title AS movie_title, m.language_code,
       sc.name AS screen_name, sh.starts_at_utc, sh.ends_at_utc
FROM theatres AS th
JOIN screens AS sc ON sc.theatre_id = th.theatre_id
JOIN showtimes AS sh ON sh.screen_id = sc.screen_id
JOIN movies AS m ON m.movie_id = sh.movie_id
WHERE th.theatre_id = @theatre_id
      AND sh.status = 'SCHEDULED'
      AND sh.starts_at_utc >= @day_start_utc
      AND sh.starts_at_utc < @day_end_utc
ORDER BY sh.starts_at_utc, m.title, sc.name;
```

See `sql/05_p2_show_query.sql` for the directly executable parameterized query.

Concurrency Test

`sql/06_concurrency_test.sql` is a two-session check. Run it concurrently in two MySQL clients, setting the test user to 2 in one session and 3 in the other. Both target show 1001, seat 403. One should report one seat acquired; the other should report zero after the conflicting allocation is observed. Reset the fixture seat between runs.

Burst Load Test

The repository contains no running API, so a measured system load result cannot be produced from the SQL-only starter. The k6 script targets the hold API contract and proves the one-winner property once connected to a backend.

```
k6 run -e BASE_URL=http://localhost:8080 \  
-e SHOW_ID=1001 -e SEAT_ID=403 -e VUS=500 \  
tests/seat-contention.js
```

Expected contract: `POST /v1/holds` returns 201 for the single winner and 409 for losing contenders. The test thresholds exactly one successful hold and less than 1% unexpected HTTP failures. Repeat with varied seats and ramped arrivals; record p95/p99 latency, throughput, lock waits, deadlocks, database CPU, Redis pressure, expiry lag, and queue depth. A same-seat burst validates exclusivity, not platform-wide capacity.

Run Order

```
mysql -u root -p < sql/01_create_database.sql  
mysql -u root -p < sql/02_create_tables.sql  
mysql -u root -p < sql/03_sample_data.sql  
mysql -u root -p < sql/04_p1_queries.sql  
mysql -u root -p < sql/05_p2_show_query.sql
```

Requires MySQL 8.0.16+ for enforced CHECK constraints. The bootstrap is non-destructive. Full operational notes and sample-row details are in `docs/database-design.md`.